

MATA54 - Estruturas de Dados e Algoritmos II

Hashing Universal e Hashing Perfeito

Flávio Assis e George Lima

IC - Instituto de Computação

Salvador, setembro de 2021

Hashing universal

O uso de uma única função de hashing pode resultar em mau desempenho de um sistema para conjuntos específicos de chaves, se a função resultar em muitas colisões (potencialmente todas as chaves podem colidir).

- ▶ Idéia: alterar a função de hashing e escolhê-la aleatoriamente. Solução de Carter e Wegman (1979).

Hashing universal

O uso de uma única função de hashing pode resultar em mau desempenho de um sistema para conjuntos específicos de chaves, se a função resultar em muitas colisões (potencialmente todas as chaves podem colidir).

- ▶ Idéia: alterar a função de hashing e escolhê-la aleatoriamente. Solução de Carter e Wegman (1979).

Hashing perfeito

Se o conjunto de chaves é conhecido, uma função hashing pode ser projetada para garantir $O(1)$ no pior caso com pouco uso de espaço

- ▶ Há métodos para se encontrar uma tal função?

Hashing universal

O uso de uma única função de hashing pode resultar em mau desempenho de um sistema para conjuntos específicos de chaves, se a função resultar em muitas colisões (potencialmente todas as chaves podem colidir).

- ▶ Idéia: alterar a função de hashing e escolhê-la aleatoriamente. Solução de Carter e Wegman (1979).

Hashing perfeito

Se o conjunto de chaves é conhecido, uma função hashing pode ser projetada para garantir $O(1)$ no pior caso com pouco uso de espaço

- ▶ Há métodos para se encontrar uma tal função?

Solução a ser estudada:

- ▶ Hashing perfeito a partir de funções hashing universais [Cormen *et al.* 2009]

Definição

Seja $\mathcal{H}$ um conjunto finito de funções hashing que mapeiam um conjunto U de chaves no conjunto $\{0, 1, \dots, m-1\}$ para um dado m . $\mathcal{H}$ é universal se, para quaisquer chaves distintas k e k^* em U , o número de funções hashing $h \in \mathcal{H}$ tais que $h(k) = h(k^*)$ é no máximo $\frac{|\mathcal{H}|}{m}$. Analogamente,

$$\forall k, k^* \in U, k \neq k^*, \Pr_{h \in \mathcal{H}} \{h(k) = h(k^*)\} \leq \frac{1}{m}$$

Exemplo de Conjunto Universal de Funções Hashing

Sejam:

- ▶ p um primo tal que toda chave possível k esteja no intervalo de 0 a $p - 1$ (inclusive) e $p > m$
- ▶ $Z_p = \{0, 1, \dots, p - 1\}$
- ▶ $Z_p^* = \{1, 2, \dots, p - 1\}$
- ▶ as funções $h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$, para $a \in Z_p^*$ e $b \in Z_p$

A família de funções de hashing

$$\mathcal{H}_{p,m} = \{h_{a,b} : a \in Z_p^* \text{ e } b \in Z_p\}$$

é universal (**Teorema 11.5 [Cormen et al. 2009]**).

Exemplo de Conjunto Universal de Funções Hashing

Seja o conjunto de chaves $U = \{10, 22, 37, 40, 52, 60, 70, 72, 75\}$ e $m = 9$.

Para $p = 101$, temos:

▶ $Z_{101} = \{0, 1, \dots, 100\}$

▶ $Z_{101}^* = \{1, 2, \dots, 100\}$

O seguinte conjunto de funções é universal:

$$h_{1,0}(k) = ((k) \bmod 101) \bmod 9$$

$$h_{1,1}(k) = ((k+1) \bmod 101) \bmod 9$$

$$h_{1,2}(k) = ((k+2) \bmod 101) \bmod 9$$

...

$$h_{1,100}(k) = ((k+100) \bmod 101) \bmod 9$$

$$h_{2,0}(k) = ((2k) \bmod 101) \bmod 9$$

$$h_{2,1}(k) = ((2k+1) \bmod 101) \bmod 9$$

$$h_{2,2}(k) = ((2k+2) \bmod 101) \bmod 9$$

...

$$h_{2,100}(k) = ((2k+100) \bmod 101) \bmod 9$$

$$h_{3,0}(k) = ((3k) \bmod 101) \bmod 9$$

$$h_{3,1}(k) = ((3k+1) \bmod 101) \bmod 9$$

...

$$h_{100,0}(k) = ((100k) \bmod 101) \bmod 9$$

$$h_{100,1}(k) = ((100k+1) \bmod 101) \bmod 9$$

...

$$h_{100,100}(k) = ((100k+100) \bmod 101) \bmod 9$$

Verificando que $\mathcal{H}_{p,m}$ é universal – Teorema 11.5 [Cormen et al. 2009]

$$\Pr\{h_{a,b}(k) = h_{a,b}(k^*)\} \leq \frac{1}{m} \text{ para } k \neq k^*$$

Sejam $r = (ak + b) \bmod p$ e $s = (ak^* + b) \bmod p$. Então,

$$r - s \equiv a(k - k^*) \pmod{p}$$

{obs: $a \neq 0$, $k \neq k^*$ e p é primo}

Então, $r - s \neq 0$. Há $p - 1$ escolhas para a e p escolhas para b tal que $r - s \neq 0$, ou seja, **há $p(p - 1)$ escolhas do par (r, s) em $\{1, \dots, p - 1\} \times \{0, 1, \dots, p - 1\}$.**

Como queremos saber probabilidade de colisão, é preciso determinar quantas escolhas de (r, s) fazem $r \equiv s \pmod{m}$?

- ▶ Para cada um dos p valores de r , há no máximo $\lceil p/m \rceil - 1$ de s tal que $s \neq r$ e $h_{a,b}(k) = h_{a,b}(k^*)$ {obs: $0 \leq h_{a,b} < m$ }. Então,

$$\Pr\{h_{a,b}(k) = h_{a,b}(k^*)\} = \frac{\lceil \frac{p}{m} \rceil - 1}{p - 1} \leq \frac{\frac{p+m-1}{m} - 1}{p - 1} = \frac{\frac{p-1}{m}}{p - 1} = \frac{1}{m}$$

Uso de hashing universal

Supondo um domínio de chaves $U = \{0, 1, \dots, \max\}$,

1. Escolher primo $p \geq |U|$.
2. Determinar o espaço de endereçamento $m < p$ adequado à aplicação
3. Escolher **aleatoriamente** valores de $a \in \{1, \dots, p-1\}$ e $b \in \{0, 1, \dots, p-1\}$
4. Definir a função escolhida $h_{a,b}$
5. Executar a aplicação com $h_{a,b}$
 - ▶ Uma vez que $h_{a,b}$ está definida, ela segue fixa
 - ▶ Mesmo que um hacker tenha acesso ao código fonte, ele não saberá como as chaves serão mapeadas nos m endereços

Definição

Um método de hashing é **perfeito** se o número de acessos é garantidamente $O(1)$ no pior caso para uma operação de busca, remoção ou inclusão.

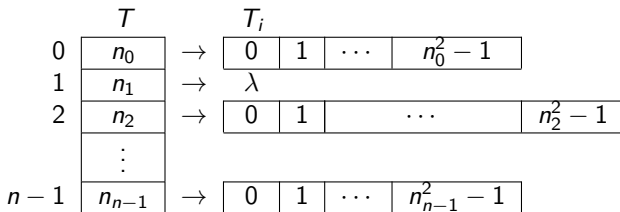
Observações

- ▶ O método a ser apresentado só se aplica a um conjunto fixo de chaves
- ▶ Aplicações: palavras reservadas em um compilador, diretórios em um sistemas de arquivos fixo (ex.: CD-ROM), palavras de um dicionário, etc..

Método: Hashing Perfeito em Dois Níveis

Idéia básica: hashing em dois níveis

- ▶ Uma função hash h de um conjunto de funções de hashing universal é usada para mapear n chaves em n endereços numa tabela T
- ▶ Usando a função h , cada entrada em T terá n_i elementos
- ▶ Funções hash h_i de um conjunto de funções hashing universal são usadas para mapear as n_i chaves mapeadas no endereço i em uma outra tabela T_i com n_i^2 endereços



Exemplo: Primeiro Nível

Usando hashing perfeito para armazenar as chaves:

$$K = \{10, 22, 37, 40, 52, 60, 70, 72, 75\}$$

para $m = 9$.

Usando $p = 101$:

$$Z_{101} = \{0, 1, 2, \dots, 100\}$$

$$Z_{101}^* = \{1, 2, \dots, 100\}$$

Classe universal de funções de hashing:

$$\mathcal{H}_{101,9} = \{h_{a,b} : a \in Z_{101}^* \text{ e } b \in Z_{101}\}$$

onde:

$$h_{a,b}(k) = ((ak + b) \bmod 101) \bmod 9$$

Exemplo: Primeiro Nível

Usando hashing perfeito para armazenar as chaves:

$$K = \{10, 22, 37, 40, 52, 60, 70, 72, 75\}$$

Usando, por exemplo, $a = 3$, $b = 42$, $p = 101$ e com $m = 9$:

$$h_{3,42}(k) = ((3k + 42) \bmod 101) \bmod 9$$

$$h(10)=0$$

$$h(22)=7$$

$$h(37)=7$$

$$h(40)=7$$

$$h(52)=7$$

$$h(60)=2$$

$$h(70)=5$$

$$h(72)=2$$

$$h(75)=2$$

0	→	10
1	→	λ
2	→	60, 72, 75
3	→	λ
4	→	λ
5	→	70
6	→	λ
7	→	22, 37, 40, 52
8	→	λ

Hashing Perfeito em Dois Níveis

Foi visto anteriormente que a probabilidade de colisão de duas chaves usando uma função de um conjunto de funções de hashing universal é menor ou igual a $1/m$.

Teorema 11.9 [Cormen et al. 2009]

Se armazenarmos n chaves em uma tabela hash de tamanho $m = n^2$ usando uma função hash h escolhida aleatoriamente de uma classe universal de funções hashing, então a probabilidade de haver colisão é menor que $1/2$.

Teorema 11.9 [Cormen et al. 2009]

Probabilidade de colisão

Se h_i pertence a uma família hashing universal e $m_i = n_i^2$, então probabilidade de colisão é inferior a $1/2$.

$X_i \equiv$ número de colisões para n_i registros em m_i endereços

$$\Pr\{X_i \geq 1\}?$$

O número de possíveis combinações de pares de chaves é n_i , tomados dois a dois. Então, a probabilidade de colisão de qualquer par de chaves é

$$E[X_i] = \binom{n_i}{2} \frac{1}{m_i} = \frac{n_i^2 - n_i}{2} \times \frac{1}{n_i^2} < \frac{1}{2}$$

Pela desigualdade de Markov, $\Pr\{X_i \geq x\} \leq \frac{E[X_i]}{x}$. Para $x = 1$,

$$\Pr\{X_i \geq 1\} < 1/2$$

Exemplo: Segundo Nível

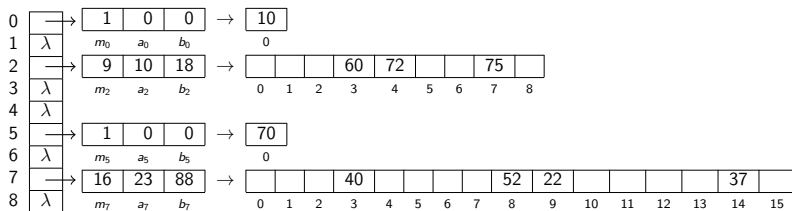
Usando hashing perfeito para armazenar as chaves:

$$K = \{10, 22, 37, 40, 52, 60, 70, 72, 75\}$$

Usando:

$$h_2(k) = ((10k + 18) \bmod 101) \bmod 9$$

$$h_7(k) = ((23k + 88) \bmod 101) \bmod 16$$



Hashing Perfeito em Dois Níveis

Hashing Perfeito em Dois Níveis

1. Escolha uma função hash da classe $\mathcal{H}_{p,n}$ tal que haja poucas colisões. Como a probabilidade de haver colisões é baixa, espera-se que poucos testes sejam necessários.
2. Para n_i chaves que são mapeadas no endereço i
 - 2.1 Escolha uma função h_i da classe universal de funções hashing $\mathcal{H}_{p,n_i^2}$
 - 2.2 Se houver colisões entre as n_i chaves usando-se a h_i escolhida, descarte h_i e repita o passo 2.1. Como a probabilidade de haver colisão é menor que $1/2$, espera-se que poucos testes sejam necessários.

Espaço para Hashing Perfeito em Dois Níveis

Quanto espaço seria necessário para usar o método descrito?

O espaço esperado de armazenamento é $O(n)$.

Teorema 11.10 [Cormen et al. 2009]

Se armazenarmos n chaves em uma tabela hash de tamanho $m = n$ usando uma função de hash h escolhida aleatoriamente de uma classe universal de funções de hashing então:

$$E \left[\sum_{i=0}^{m-1} n_i^2 \right] < 2n$$

Teorema 11.10 [Cormen et al. 2009]

Considerando $m = n$, temos que calcular $E \left[\sum_{i=0}^{m-1} n_i^2 \right]$

Seja C_{ij} uma variável aleatória indicadora: $C_{ij} = 1$ se k_i colide com k_j e $C_{ij} = 0$ caso contrário.

$$E \left[\sum_{i=0}^{m-1} n_i^2 \right] = E \left[\sum_{i=0}^{m-1} \sum_{j=0}^{m-1} C_{ij} \right] = E \left[\sum_{i=0}^{m-1} C_{ii} + \sum_{i=0}^{m-1} \sum_{j=0, j \neq i}^{m-1} C_{ij} \right]$$

Como $\Pr\{C_{ii} = 1\} = 1$ e para $i \neq j$, $\Pr\{C_{ij} = 1\} \leq 1/m$ (**hashing universal**) e adicionando o fato que $n = m$,

$$E \left[\sum_{i=0}^{m-1} C_{ii} \right] + E \left[\sum_{i=0}^{m-1} \sum_{j=0, j \neq i}^{m-1} C_{ij} \right] = n + \frac{n(n-1)}{m} = 2n - 1 < 2n$$

Espaço para Hashing Perfeito em Dois Níveis

Adicionalmente, a probabilidade de se precisar de mais do que $4n$ é menor que $1/2$.

Corolário 11.2 [Cormen et al.]

Se armazenarmos n chaves em uma tabela hash de tamanho $m = n$ usando uma função de hash h escolhida aleatoriamente de uma classe universal de funções de hashing e estabelecermos o tamanho de cada tabela secundária como sendo $m_j = n_j^2$ para $j = 0, 1, 2, \dots, m - 1$ então a probabilidade de que o total de armazenamento necessário para as tabelas hash secundárias ultrapasse $4n$ é menor do que $1/2$.

Corolário 11.2 [Cormen et al.]

Temos que calcular um limite para

$$\Pr \left\{ \sum_{i=0}^{m-1} n_i^2 \geq 4n \right\}$$

Pela desigualdade de Markov, para uma variável aleatória X , $\Pr\{X \geq x\} \leq \frac{E[X]}{x}$.

Portanto, para $X = \sum_{i=0}^{m-1} n_i^2$ e $x = 4n$,

$$\Pr \left\{ \sum_{i=0}^{m-1} n_i^2 \geq 4n \right\} \leq \frac{E \left[\sum_{i=0}^{m-1} n_i^2 \right]}{4n} < \frac{2n}{4n} = 1/2$$

Hahsing perfeito

Destes resultados conclui-se que escolhendo funções hashing de um conjunto universal de funções, é muito provável que, após algumas tentativas, chega-se a um configuração de hashing perfeito com que consuma $O(n)$ espaço e com garantias de desempenho em número de acessos de $O(1)$ (no pior caso)